STRINGS, LISTS & MUTABILITY

INFO1 Mission 4 (S5)

Jean-Luc Bibaud



Methods on strings (and other objects)

1. A string is an **object**
2. Methods are functions that come **with** the object
3. You apply them with a `.` `object.method()`
4. Lists have methods too: `nums.append(4)`

```
my_str = "hi"
my_str_upper = my_str.upper()
print(my_str_upper)
→ HI

"banana".count("a")
→ 3
"a,b,c".split(",")
→ ['a', 'b', 'c']
"hi".replace("h", "p")
→ 'pi'
```

Indexing operator

How it works

`VARIABLE[INDEX]` or **`VALUE[INDEX]`** gives a value

VALUE[...] is less common

Indexes start at 0.

What it does depends on the type:

```
# string
"hi"[0]
→ 'h'    (a str)
```

```
nums = [1, 2, 4]
nums[1]
→ 2    (an int)
```

What is iterable? How to loop through all elements?

Iterable = something you can go through one element at a time: a string, a list, a range…

```
fruit = "banana"

list(enumerate(fruit))
→ [(0, 'b'), (1, 'a'), (2, 'n'),
  (3, 'a'), (4, 'n'), (5, 'a')]

list(enumerate(range(0, 5)))
→ [(0, 0), (1, 1), (2, 2), (3, 3), (4, 4)]
```

What does this do then?

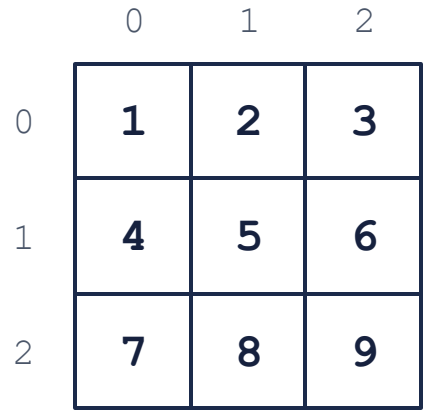
```
for i in fruit:
    print(i)
```

```
→ b
  a
  n
  a
  n
  a
```

Nested lists are possible

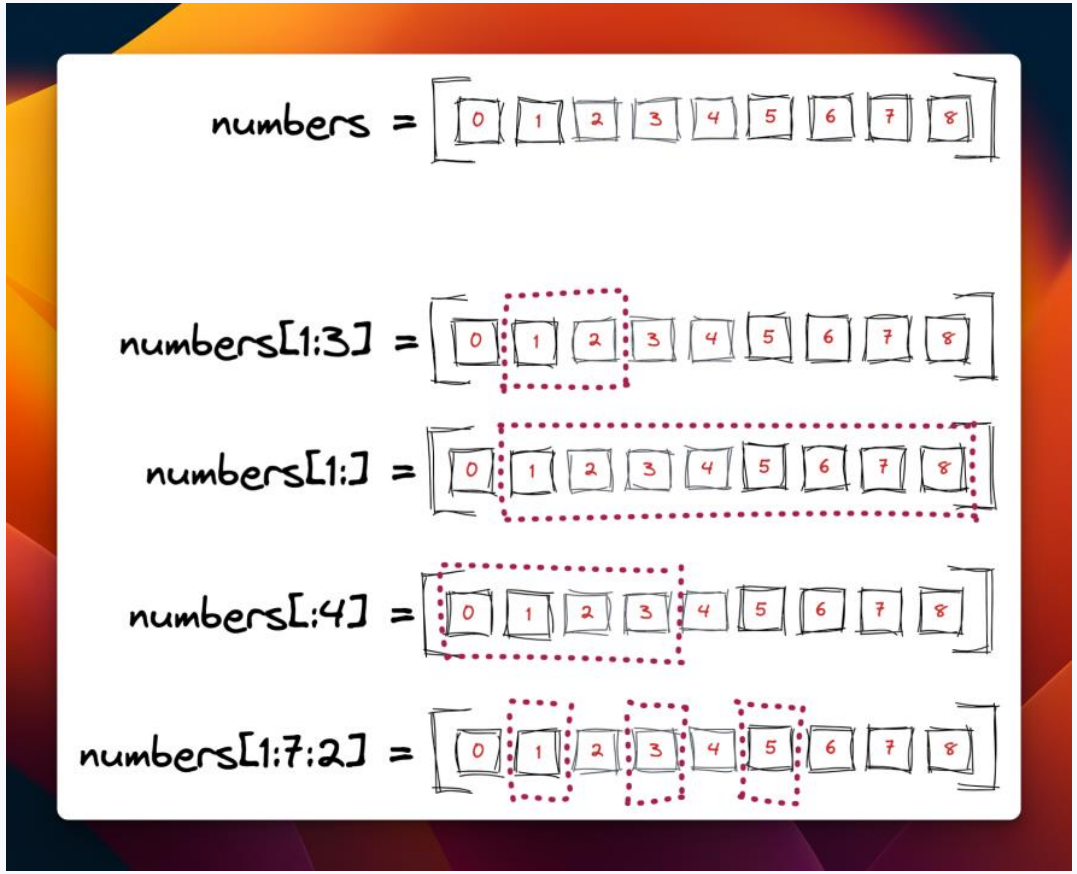
```
mx = [[1, 2, 3],
      [4, 5, 6],
      [7, 8, 9]]

for row in mx:
    print(row)
→ [1, 2, 3]
  [4, 5, 6]
  [7, 8, 9]
```




```
mx[1]       → [4, 5, 6]
mx[1][2]    → 6
mx[-1][0]   → 7
```

Slices and substrings



`[start : stop : step]` — stop is excluded

Using negative indexes?

	0	1	2	3	4	5	6
word	a	m	a	z	i	n	g
	-7	-6	-5	-4	-3	-2	-1

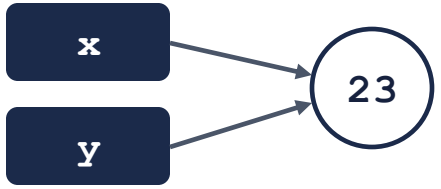
What does this give?

```
word[-1]        → 'g'
word[-3:]       → 'ing'
word[:-3]       → 'amaz'
word[-5:-2]     → 'azi'
word[::-1]      → 'gnizama'
```

Numbers and strings (and tuples) are immutable!

Numbers

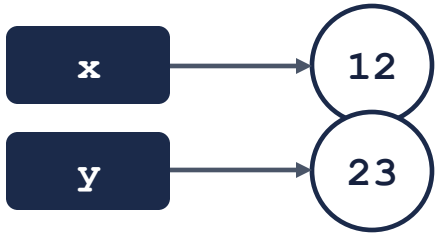
```
x = 23
y = x
```



How many distinct values exist?

```
→ 1
```

```
x = 12
```



`x = 12` rebinds the name x to the value 12. It does **not** change 23 — y still points to it.

Strings

```
greeting = "Hello, world!"
greeting[0] = "J"
→ TypeError: 'str' object does not
  support item assignment
print(greeting)
→ Hello, world!
```

Concatenation creates a new string and binds the name to that new value:

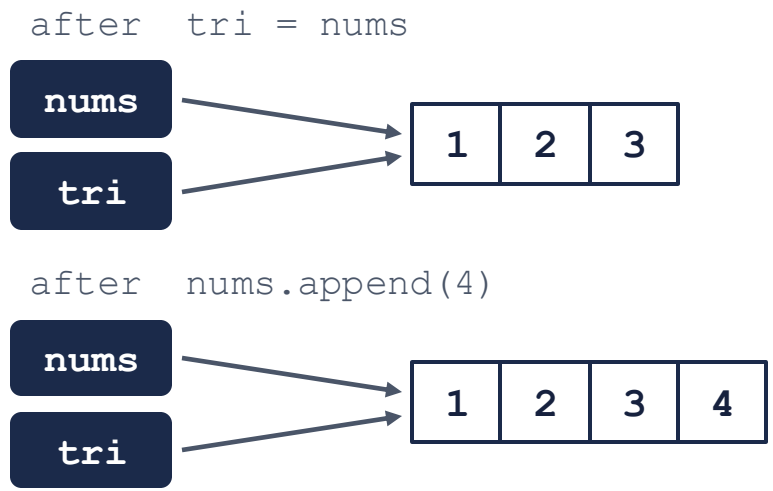
```
greeting = greeting + " Bye!"
print(greeting)
→ Hello, world! Bye!
```

Lists are mutable! Looking at a list

```
nums = [1, 2, 3]
tri = nums

nums.append(4)

print(tri)
→ [1, 2, 3, 4]
```



Only **one** list: tri is just a second name for it.

Want a separate list? `tri = nums.copy()` (or `nums[:]`)

Functions reminder

```
def NAME(PARAMETERS):
    STATEMENTS
    return VALUE    # optional
```

Without a return, the function gives back None.

Invoking / calling

= executing the code of that function

```
NAME(ARGUMENTS)
```

```
x = myfunction(20)    # x is what the function returned
```

List in a function

A list is changed in place

```
def add(lst):
    lst.append("B")
    return lst

my_list = ["X"]
add(my_list)
print(my_list)
→ ['X', 'B']
```

A number is not

```
def add(num):
    num += 1
    return num

my_num = 3
add(my_num)
→ returns 4 (not stored!)
print(my_num)
→ 3
```

The function gets the same list, so append changes it. num += 1 makes a new number, only inside.

NAMING

Two hard things

> There are 2 hard problems in computer science: cache invalidation, naming things, and off-by-1 errors.
>
> *A joke built on Phil Karlton's quote (his original only had the first two).*

All of these are assignments

```
X = ...                    with ... as X:

for X in ...               except ... as X:

[... for X in ...]         import X

(... for X in ...)         from ... import X

{... for X in ...}         import ... as X

class X(...):              from ... import ... as X

def X(...):

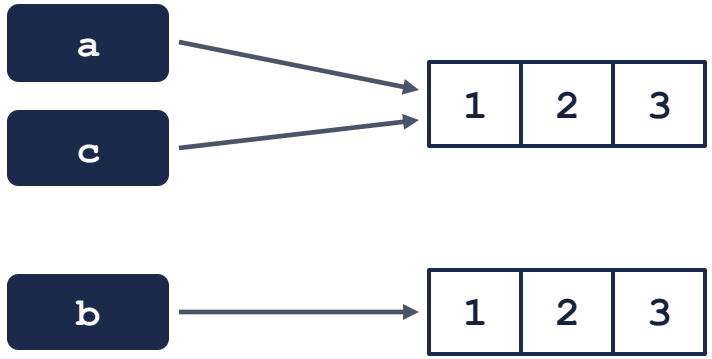
def fn(X): ... ; fn(12)
```

COMPARISON

Difference between is and ==?

== same value?

is same object (the same list in memory)?



same values, but two different lists

Use **is** only for None: **if x is None:**

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a

a == b      → True
a is b      → False
a is c      → True
```

WATCH OUT

Common beginner mistakes

Storing what append gives

```
nums = [1, 2]
nums = nums.append(3)
# nums is now None
```

append changes the list and returns None.

Thinking strings change

```
s = "hi"
s.upper()
print(s)     # hi
```

Strings never change: write s = s.upper().

Index one too far

```
word = "hi"
word[2]    # IndexError
```

The last index is len(word) - 1, or just -1.

Forgetting stop is excluded

```
"amazing"[0:3]
# 'ama', not 'amaz'
```

[start:stop] stops just before stop.

Copying a list with =

```
tri = nums
tri.append(9)
# nums changed too!
```

Use nums.copy() for a real copy.

is instead of ==

```
if name is "Ada":
# works... sometimes
```

== for values, is only for None.